AI Engineering

Revision Cards

12 cards · Foundations → Backend Migration

01 Foundations	**02** First Request	**03** Tokens & Prompts	**04** Messages
05 UI & Errors	**06** Markdown Safety	**07** Streaming	**08** Few-Shot
09 Temperature	**10** JSON/Schema	**11** Web Search	**12** Backend

Each card: explanation · examples · code · checklist · self-test

01 AI Engineering Foundations

Models · Environment · API Keys · First Setup

WHAT IS AI ENGINEERING?

AI engineering is building real products using AI models — not just chatting with them. You pick a model, write prompts, wire up a UI, and ship something useful.

Just a user
Typing prompts in a chat box. No code. No control over output.

AI Engineer
Writing code that calls the model. Controlling output, format, cost.

THE 3 TRADEOFFS WHEN PICKING A MODEL

Quality
How accurate and smart the answers are. Better model = better output.

Speed
How fast the model responds. Faster = better UX for users.

Cost
How much per API call. More tokens + bigger model = more expensive.

ENVIRONMENT SETUP — WHY IT MATTERS

Wrong keys or URLs break everything silently. Always store them in a .env file, never hardcode them.

env + client setup

```
# .env  — never commit this file to git
AI_URL=https://api.anthropic.com
AI_MODEL=claude-sonnet-4-20250514
AI_KEY=sk-ant-...


import OpenAI from 'openai'
const client = new OpenAI({
  baseURL: process.env.AI_URL,
  apiKey:  process.env.AI_KEY,
})
```

- ■ `AI_URL set in .env`
- ■ `AI_MODEL set in .env`
- ■ `AI_KEY set in .env`
- ■ `Client initialised without errors`

✎ Self-test

1. What is the difference between using AI and engineering with AI?
2. Why use environment variables instead of writing the key directly in code?
3. Name one tradeoff you'd make for a fast, cheap chatbot.

02 Your First AI Request

Input → Payload → Call → Output → Display

THE CORE FLOW — 4 STEPS

1 →	2 →	3 →	4
Collect input	**Build payload**	**Call model**	**Show output**
Get user's text	Add model + messages	Await the response	Render the result

WHAT MAKES A GOOD PROMPT?

✗ Bad

```
Give me gift ideas
```

✓ Good

```
Suggest 3 birthday gift ideas for a
25-year-old who loves cooking.
Budget: under £30. UK-based.
```

- ✓ Audience is clear
- ✓ Budget is defined
- ✓ Format is predictable
- ✓ Location is set

MINIMAL WORKING REQUEST

simplest possible call

```
const res = await client.chat.completions.create({
  model:   process.env.AI_MODEL,
  messages: [{ role: 'user', content: userPrompt }],
  max_tokens: 500,
})

// Extract the text
const output = res.choices[0].message.content
console.log(output)
```

COMMON MISTAKE

✗ Bad

```
const output = res.content
// Wrong — this is undefined
```

✓ Good

```
const output = res.choices[0].message.content
// Correct path
```

- ■ User input collected
- ■ Payload has model + messages
- ■ Correct response path used
- ■ Output displayed or logged

✎ **Self-test**

Rewrite this prompt with 3 improvements:

"Write about space" → add audience + format + length

03 Prompt Refinement & Token Control

Clearer prompts · Cheaper responses · Faster AI

PART 1 — PROMPT REFINEMENT

Rewrite your prompt so AI knows exactly what you want: level, format, length, audience.

✗ Bad

```
Explain JavaScript
```

✓ Good

```
Explain JavaScript closures to a beginner.
Use 3 bullet points. No jargon.
```

✗ Bad

```
Write about AI
```

✓ Good

```
Explain AI in 5 lines for a 12-year-old.
Use simple words. Bullet points. No jargon.
```

- ✓ More accurate answers
- ✓ Consistent format
- ✓ Less hallucination
- ✓ Cheaper (fewer retries)

PART 2 — TOKEN CONTROL

AI counts tokens (word pieces), not words. More tokens = more cost + slower response. Tokens ≈ ¾ of a word. 'Hello world' ≈ 3 tokens.

1	2	3	4
Short output	**Limit bullets**	**API hard limit**	**Be specific**
Explain in 3 lines	Give 5 bullets only	max_tokens: 100	In 2 sentences max

BOTH SKILLS COMBINED

```
Explain async/await in JavaScript

for beginners

in 4 bullet points

under 50 words total
```

- ■ Audience defined
- ■ Format specified
- ■ Length constrained
- ■ max_tokens set in API call
- ■ No vague open-ended questions

✎ **Self-test**

Rewrite: "Tell me about databases"

→ Add: who it is for + format + word/bullet limit

04 Messages Array & System Message

Memory · Personality · Rules · Conversation Flow

WHAT IS THE MESSAGES ARRAY?

AI has no memory between calls. You must send the full conversation every time. The messages array is that conversation — a list of who said what, in order.

```
messages = [

  { role: "system",    content: "You are a helpful teacher" },

  { role: "user",      content: "What is gravity?" },

  { role: "assistant", content: "Gravity pulls things down" },

  { role: "user",      content: "Why does it pull?" }

]
```

THE 3 ROLES — EXPLAINED SIMPLY

system
Controls AI behaviour. Tone, rules, format. Runs silently. Most powerful.

user
The human's current message. What you actually ask or type.

assistant
The AI's previous reply. Keeps the conversation flowing naturally.

SYSTEM MESSAGE EXAMPLES

Teacher mode `"You are a friendly math teacher"`	Strict mode `"Answer in one sentence only"`
Coder mode `"You are a senior JS engineer"`	Topic lock `"You only answer about cooking"`

Simple analogy

- Messages array = classroom conversation
- system = teacher instructions
- user = student question
- assistant = teacher answer

Why it matters

- Without messages array — AI forgets everything
- With it — AI remembers and stays in character
- system is the most powerful message

- `system message defined`
- `user message is dynamic`
- `assistant history included when needed`
- `System sets tone + rules`

✎ **Self-test**

Write a system message that makes AI:

1. Talk like a 5-year-old explainer
2. Answer in max 3 sentences
3. Refuse non-coding questions

05 UI Wiring & Error Handling

Submit → Loading → Request → Render → Reset

THE SUBMIT FLOW — STEP BY STEP

1 →	2 →	3 →	4 →	5
Capture	**Disable**	**Fetch**	**Render**	**Reset**
e.preventDefault()	btn.disabled = true	POST to /api/gift	Display output	Re-enable button

THREE PLACES ERRORS CAN HAPPEN

Before request
Invalid input, empty prompt, missing fields. Validate before calling.

During request
Network failure, non-OK HTTP status, timeout. Wrap in try/catch.

While rendering
Model returns unexpected format. Safely parse before inserting into DOM.

FULL ERROR-SAFE SUBMIT HANDLER

complete submit handler

```
async function handleSubmit(e) {
  e.preventDefault()
  btn.disabled = true
  output.textContent = ''
  try {
    const res = await fetch('/api/gift', {
      method: 'POST',
      body: JSON.stringify({ userPrompt }),
    })
    if (!res.ok) throw new Error(res.statusText)

    const data = await res.json()

    render(data.giftSuggestions)

  } catch (err) {

    showError('Something went wrong — try again.')

    console.error(err)

  } finally {

    btn.disabled = false   // ALWAYS re-enable

```

```
  }

}
```

- e.preventDefault() called
- Submit disabled during call
- if (!res.ok) throws error
- Error shown to user (not just console)
- finally re-enables button

✎ **Self-test**

What happens if you forget finally { btn.disabled = false }?

What should you show the user vs. log to console?

06 Rendering Markdown Safely

marked → DOMPurify → innerHTML — always in this order

THE GOLDEN RULE

■ **Never inject raw model output directly into the DOM. It could contain malicious HTML.**

WHY IS MODEL OUTPUT DANGEROUS?

✗ Bad

```
output.innerHTML = modelText
// DANGEROUS — raw injection
```

✓ Good

```
const html  = marked.parse(modelText)
const clean = DOMPurify.sanitize(html)
output.innerHTML = clean  // Safe ✓
```

THE SAFE RENDER PIPELINE

1 →	2 →	3 →	4
Model text	**marked.parse**	**DOMPurify**	**innerHTML**
Raw markdown string	Convert → HTML	Remove dangerous tags	Insert safely

FULL SAFE RENDER FUNCTION

safe render function

```
import { marked }  from 'marked'
import DOMPurify   from 'dompurify'

function render(markdownText) {
  const rawHtml   = marked.parse(markdownText)
  const cleanHtml = DOMPurify.sanitize(rawHtml)
  output.innerHTML = cleanHtml
}

// Rule: parse FIRST, sanitize SECOND, never reverse.
```

- ■ marked installed
- ■ DOMPurify installed
- ■ Parse markdown before sanitizing
- ■ Sanitize before innerHTML
- ■ Never use raw model text as HTML

✎ **Self-test**

Why must sanitize come AFTER marked.parse, not before?

What could go wrong if you skip DOMPurify?

07 Streaming Responses

Chunks arrive live · Better UX · for-await loop

STREAMING vs WAITING — THE DIFFERENCE

Without streaming

- Wait for full response (default)
- User sees nothing for 3-5 seconds
- Then entire text appears at once
- Feels slow and unresponsive

With streaming

- Streaming mode
- Text appears word by word instantly
- User sees progress immediately
- Feels fast, alive, engaging

STREAMING WITH for-await

streaming with for-await

```
const stream = await client.chat.completions.create({
  model:    process.env.AI_MODEL,
  messages,
  stream:   true,   // Enable streaming
})

for await (const chunk of stream) {
  const delta = chunk.choices[0]?.delta?.content ?? ''
  output.textContent += delta  // Append, not replace!
}
```

KEY RULES FOR STREAMING

Append, don't replace
Use += not =. Each chunk adds to the output, never overwrites it.

Optional chaining ?.
Use chunk.choices[0]?.delta?.content to avoid crashes on empty chunks.

?? '' fallback
Default to empty string when delta is undefined — avoids showing 'undefined'.

Finalize after stream
Reset loading state AFTER the stream loop completes, not inside it.

- `stream: true` set
- Chunks appended (+=)
- Optional chaining used
- `?? ''` fallback present
- State reset after loop ends

✎ **Self-test**

What happens if you use = instead of += for chunks?

Why is streaming 'better feel' but not 'better quality'?

08 Context-Aware & Few-Shot Prompting

Give context · Show examples · Get consistent output

CONTEXT-AWARE PROMPTING

Include real constraints the model can act on. Vague prompts get vague answers.

✗ Bad

```
Give me gift ideas
```

✓ Good

```
Budget: £30. Recipient: keen baker.
Location: UK. Occasion: birthday.
Suggest 3 ideas with reasons.
```

Who	Budget	Where	When
audience / recipient	price constraint	location / region	urgency / occasion

FEW-SHOT PROMPTING — TEACH BY EXAMPLE

Give 1–3 example input/output pairs before your real question. AI copies the pattern. Keeps output consistent every time.

few-shot setup

```
const FEW_SHOT = [
  {
    role: "user",
    content: "Budget £30, recipient: keen baker, UK"
  },
  {
    role: "assistant",
    content: "## Silicone Baking Mat\n**Why:** Practical..."
  },
]



// Prepend FEW_SHOT before the actual user message

messages = [...FEW_SHOT, { role:'user', content: userPrompt }]

```

Good examples

- Examples are short (1-3 max)
- Realistic — same domain as your app
- Match your target output format

Pitfalls

- Too many examples wastes tokens
- Bad examples teach bad patterns
- Examples still count toward token cost

✎ **Self-test**

Write one few-shot example pair for a recipe recommender:

input: diet + cuisine → output: structured markdown recipe name + reason

09 Temperature, Top P & Model Choice

Predictability ↔ Creativity · Snapshot versions

TEMPERATURE — THE CREATIVITY DIAL

Low temperature
- temperature: 0.0 → 0.3
- Very predictable, same answer every time
- Best for: factual Q&A;, production apps, tests

High temperature
- temperature: 0.7 → 1.0
- More creative, varied, surprising answers
- Best for: brainstorming, stories, ideas

TOP P — THE OTHER DIAL (USE ONLY ONE)

Rule: tune temperature OR top_p — never both aggressively at the same time.

top_p: 0.1
Only considers the top 10% of likely words. Very focused.

top_p: 0.9
Considers the top 90% of words. More varied. Good default.

TUNING ORDER — ALWAYS DO THIS SEQUENCE

1 →	2 →	3 →	4
Improve prompt	**Adjust temp**	**Try top_p**	**Switch model**
Fix instructions first	Try 0.3 or 0.7	Only if needed	Last resort only

temperature in practice

```
// Production recommender — start conservative
const res = await client.chat.completions.create({
  model:       'claude-sonnet-4-20250514',
  messages,
  temperature: 0.3,    // predictable, consistent
  // top_p: 0.9,       // tune one OR the other
  max_tokens:  600,
})
```

- ■ Prompt refined before tuning params
- ■ Temperature chosen intentionally
- ■ Not tuning temperature + top_p together
- ■ Model snapshot version pinned

✎ **Self-test**

Gift recommender — high or low temperature? Why?

Creative story generator — what temperature would you start with?

10 JSON Mode & Structured Outputs

Free text → JSON mode → Strict schema — pick the right tool

THREE OUTPUT MODES — WHEN TO USE EACH

Free text (default)
Quick prototypes. Easy to read. Hard for your code to parse reliably.

JSON mode
Frontend needs to parse the output. Tells AI to respond only with valid JSON.

Structured outputs
Strict production contracts. AI must match an exact schema you define.

JSON MODE vs STRUCTURED OUTPUT — CODE COMPARISON

json mode vs structured schema

```
// Option 1 — JSON mode (less strict)
{ response_format: { type: 'json_object' } }

// Option 2 — Structured output (strict schema)
{ response_format: {
    type: 'json_schema',
    json_schema: {
      name: 'gift_response',
      strict: true,
      schema: {
        type: 'object',
        properties: {
          gifts: { type: 'array' },
          count: { type: 'number' }
        }
      }
    }
  }
}
```

SAFELY PARSING JSON RESPONSES

safe json parse

```
try {
  const text   = data.content[0].text
  const clean  = text.replace(/```json|```/g, '').trim()
  const parsed = JSON.parse(clean)
  render(parsed.gifts)
} catch (err) {
  showError('Unexpected response format')
}
```

- [ ] Mode chosen for use case
- [ ] Schema defined when using structured outputs
- [ ] JSON parsed inside try/catch
- [ ] Schema versioned for production

Self-test

When should you use strict schema instead of JSON mode?

What could go wrong if you skip the try/catch on JSON.parse?

11 Web Search Tool Integration

Fresh data · Location-aware · Grounded answers

WHY ADD WEB SEARCH?

Without search

- Model has a knowledge cutoff date
- Generic answers — no local context
- Can't know today's prices or availability

With search

- Search gives real-time results
- Location-aware recommendations
- Grounded in actual current facts

WHEN TO USE SEARCH — AND WHEN NOT TO

Use search ✓
Current prices, local stores, today's news, live product availability.

Skip search ✗
Timeless facts, code patterns, general explanations — no need for search.

Risk to avoid
Unnecessary search adds latency + cost. Only trigger it when live facts matter.

ADDING WEB SEARCH TO YOUR API CALL

web search tool config

```
const res = await client.chat.completions.create({
  model:    'claude-sonnet-4-20250514',
  messages,
  tools: [{
    type: 'web_search_20250305',
    name: 'web_search',
  }],
})

// AI decides when to search based on the query.

// Merge retrieved facts into the final answer.
```

- ■ Search only added when live facts matter
- ■ User intent is clear before searching
- ■ Latency impact considered
- ■ Retrieved facts merged into final answer

✎ Self-test

Give one prompt where search IS essential.

Give one prompt where search would just slow things down.

12 Backend Orientation & Migration

Security · API keys server-side · Centralised control

WHY MOVE AI CALLS TO THE BACKEND?

Security
API key stays on the server. Never exposed in browser DevTools or source code.

Logging
One central place to log every request, response, error, and cost.

Rate limiting
Control how many calls per user. Prevent abuse. Set per-user quotas.

Policy
Enforce output rules, filtering, and schema validation in one place.

THE FULL REQUEST PATH

1 →	2 →	3 →	4 →	5
Frontend	**Backend**	**Model**	**Backend**	**Frontend**
POST /api/gift	Read body	Call via SDK	Return JSON	Render result

frontend + backend split

```
// ■■ Frontend ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const res  = await fetch('/api/gift', {
  method: 'POST',
  body:   JSON.stringify({ userPrompt }),
})
render((await res.json()).giftSuggestions)

// ■■ Backend (Express) ■■■■■■■■■■■■■■■■■■■■■■■■■■■
app.post('/api/gift', async (req, res) => {
  const { userPrompt } = req.body

  messages.push({ role:'user', content: userPrompt })

  const r = await client.chat.completions.create({...})

  res.json({ giftSuggestions: r.choices[0].message.content })

})
```

- Frontend POSTs userPrompt
- Backend reads req.body
- Model called server-side only

```
Key never in client code
```

✎ Self-test

If frontend expects data.giftSuggestions,
what exactly must the backend return?
What breaks if the field name doesn't match?